



---

# Report Project 2

---

LEGGED ROBOTS  
PROFESSOR : AUKE IJSPEERT

LJUNGBERG Oliver  
VRAY Alexandre  
ODDON Rémi

Group 25

Autumn 2022

## Table des matières

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                            | <b>3</b>  |
| <b>2</b> | <b>Quadruped Modeling</b>                      | <b>3</b>  |
| <b>3</b> | <b>Central Pattern Generator</b>               | <b>4</b>  |
| 3.1      | Implementation . . . . .                       | 4         |
| 3.2      | Trot gait . . . . .                            | 5         |
| 3.2.1    | CPG states . . . . .                           | 5         |
| 3.2.2    | Foot trajectory . . . . .                      | 6         |
| 3.2.3    | Joint angles trajectory . . . . .              | 6         |
| 3.2.4    | Discussion of hyperparameters tuning . . . . . | 7         |
| 3.2.5    | Locomotion videos . . . . .                    | 8         |
| 3.2.6    | Discussion of controller extension . . . . .   | 8         |
| 3.3      | Walk gait . . . . .                            | 9         |
| 3.3.1    | CPG states . . . . .                           | 9         |
| 3.3.2    | Foot trajectory . . . . .                      | 10        |
| 3.3.3    | Joint angles trajectory . . . . .              | 10        |
| 3.3.4    | Hyperparameters tuning . . . . .               | 11        |
| 3.3.5    | Locomotion videos . . . . .                    | 12        |
| 3.4      | Pace gait . . . . .                            | 12        |
| 3.4.1    | CPG states . . . . .                           | 12        |
| 3.4.2    | Foot trajectory . . . . .                      | 13        |
| 3.4.3    | Joint angles trajectory . . . . .              | 14        |
| 3.4.4    | Hyperparameters tuning . . . . .               | 15        |
| 3.4.5    | Locomotion videos . . . . .                    | 15        |
| 3.5      | Bound gait . . . . .                           | 16        |
| 3.5.1    | CPG states . . . . .                           | 16        |
| 3.5.2    | Foot trajectory . . . . .                      | 17        |
| 3.5.3    | Joint angles trajectory . . . . .              | 17        |
| 3.5.4    | Hyperparameters tuning . . . . .               | 18        |
| 3.5.5    | Locomotion videos . . . . .                    | 19        |
| <b>4</b> | <b>Deep Reinforcement Learning</b>             | <b>19</b> |
| 4.1      | Action space . . . . .                         | 19        |
| 4.2      | Observation space . . . . .                    | 19        |
| 4.3      | Reward function . . . . .                      | 20        |
| 4.4      | DRL algorithm . . . . .                        | 20        |
| 4.5      | Environmental details . . . . .                | 20        |
| 4.6      | Approach robustness . . . . .                  | 21        |
| 4.7      | Resulting control policy using PPO . . . . .   | 21        |
| 4.7.1    | Reward and Episode length . . . . .            | 21        |
| 4.7.2    | CPG states . . . . .                           | 22        |
| 4.7.3    | Foot trajectory . . . . .                      | 23        |
| 4.8      | Resulting control policy using SAC . . . . .   | 24        |
| 4.8.1    | Reward and Episode length . . . . .            | 24        |
| 4.8.2    | CPG states . . . . .                           | 25        |

|          |                             |           |
|----------|-----------------------------|-----------|
| 4.8.3    | Foot trajectory . . . . .   | 26        |
| 4.9      | Locomotion videos . . . . . | 27        |
| <b>5</b> | <b>Conclusion</b>           | <b>27</b> |

## 1 Introduction

The goal of this project is to find a way to implement a robust method of locomotion for a quadruped robot, that hopefully could resist perturbations such as irregular terrain, added weight on the back of the robot, and so on. For this purpose, we will first show how to implement a model of the actuators and the joint space of the robot, then how to achieve locomotion with Central Pattern Generators, and finally how to achieve locomotion with Deep Reinforcement Learning, by training the model with Markov Decision Process algorithms. The goal is to find the advantages and inconvenience of each of these model to see if it is better to let an algorithm take care of the motion planning or to implement it by hand.

## 2 Quadruped Modeling

The quadruped robot we are using has 3 actuated joints on each of its 4 legs, for a total of 12 actuators. Two actuators are positioned on each hip and one on each knee, to simulate the actuation of the hip, the thigh and calf as presented in figure 1.

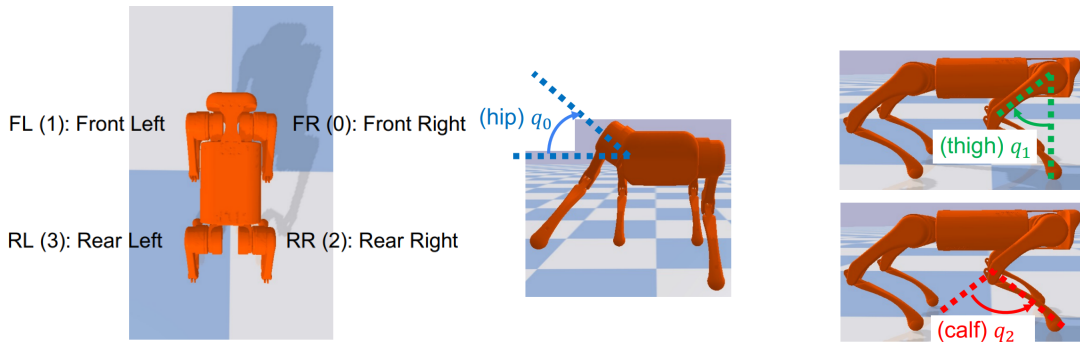


FIGURE 1 – Actuator position and numeration

Each leg will have its forward kinematics expressed in its own frame, so we can compute the relative position of the foot  $\mathbf{p}$  from the joint angles  $\mathbf{q}$ . We can then easily express  $\mathbf{p}$  as :

$$\mathbf{p} = \mathbf{f}(\mathbf{q})$$

Then, by deriving the function  $\mathbf{f}(\cdot)$ , we can compute the Jacobian  $\mathbf{J}(\cdot)$ , thanks to which we can find the velocity of the feet as a function of the joint angles and joint velocities :

$$\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

We also know an expression that links the torques  $\boldsymbol{\tau}$  on the actuators, given the forces  $\mathbf{F}$  applied on them :

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q}) \cdot \mathbf{F}$$

Two controllers are chosen for this project, and both are PD controllers. The first one,  $\boldsymbol{\tau}_{Joint}$ , is given in the joint space, and is expressed as a function of the proportional and derivative controller gain matrices  $\mathbf{K}_{p,Joint}$  and  $\mathbf{K}_{d,Joint}$ , that are respectively multiplied by the difference between the desired joint position  $\mathbf{q}_d$  and current position  $\mathbf{q}$ , and between the desired joint velocity  $\dot{\mathbf{q}}_d$  and current velocity  $\dot{\mathbf{q}}$ . The second one,  $\boldsymbol{\tau}_{Cartesian}$ , is given in the leg frame and will therefore use the foot position and velocity  $\mathbf{p}$  and  $\dot{\mathbf{p}}$  instead of  $\mathbf{q}$

and  $\dot{\mathbf{q}}$ , and both  $\mathbf{K}_p$  and  $\mathbf{K}_d$  are expressed in the Cartesian frame instead. Their equation can be expressed as :

$$\begin{aligned}\boldsymbol{\tau}_{Joint} &= \mathbf{K}_{p,Joint} \cdot (\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_{d,Joint} \cdot (\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) \\ \boldsymbol{\tau}_{Cartesian} &= \mathbf{J}^T(\mathbf{q}) \left( \mathbf{K}_{p,Cartesian} \cdot (\mathbf{p}_d - \mathbf{p}) + \mathbf{K}_{d,Cartesian} \cdot (\dot{\mathbf{p}}_d - \dot{\mathbf{p}}) \right)\end{aligned}$$

### 3 Central Pattern Generator

#### 3.1 Implementation

The equations used for CPG are Hopf oscillators, where each leg is considered as an oscillator indexed with  $i$ . The amplitude is given by  $r_i$  and its phase by  $\theta_i$ . We will call  $\omega_i$  the natural frequency of the oscillator and  $\sqrt{\mu}$  the desired amplitude. We also have to define the coupling strength between the two oscillators  $i$  and  $j$  (called  $w_{i,j}$ ) and the desired phase offset between these oscillators (called  $\phi_{i,j}$  to be able to simulate different gates. Then we can define the two following equations :

$$\begin{aligned}\dot{r}_i &= \alpha \cdot (\mu - r_i^2) \cdot r_i \\ \dot{\theta}_i &= \omega_i + \sum_{j=0}^3 r_j \cdot w_{i,j} \cdot \sin(\theta_j - \theta_i - \phi_{i,j})\end{aligned}$$

with  $\alpha$  a positive constant that controls the speed of convergence to the limit cycle.

The Hopf oscillators are defined to have two different states. When the leg is in contact with the ground, it is in stance state. The rest of the time, when the leg isn't touching the ground, the leg is in swing state. The swing state will have a different natural frequency  $\omega_{i,swing}$  than the stance state ( $\omega_{i,stance}$ ), and  $\omega_i$  will alternate between both frequencies depending on the state. The state is chosen according to the current phase of the oscillator, and we define that the swing phase is occurring for  $\theta_i \in [0; \pi]$ , and the leg is in stance phase for  $\theta_i \in [\pi; 2\pi]$ . We will define the duration for the swing phase and stance phase  $T_{swing}$  and  $T_{stance}$ , as well as the duty ratio  $DasD = T_{stance}/T_{swing}$ , and we will also define  $g_c$  as the maximum ground clearance during the swing, and  $g_p$  the maximum ground penetration during the stance. We can then finally express the position of the foot in the leg frame along the x-axis and the z-axis, (making the hypothesis that the position along the y-axis is always 0) as follows :

$$\begin{aligned}x_i &= -d_{step} \cdot r_i \cdot \cos(\theta_i) \\ z_i &= -h + g \cdot \sin(\theta_i)\end{aligned}$$

with  $d_{step}$  the step length,  $h$  the robot height, and  $g$  being either  $g_c$  or  $g_p$  depending on the current state of the oscillator.

## 3.2 Trot gait

### 3.2.1 CPG states

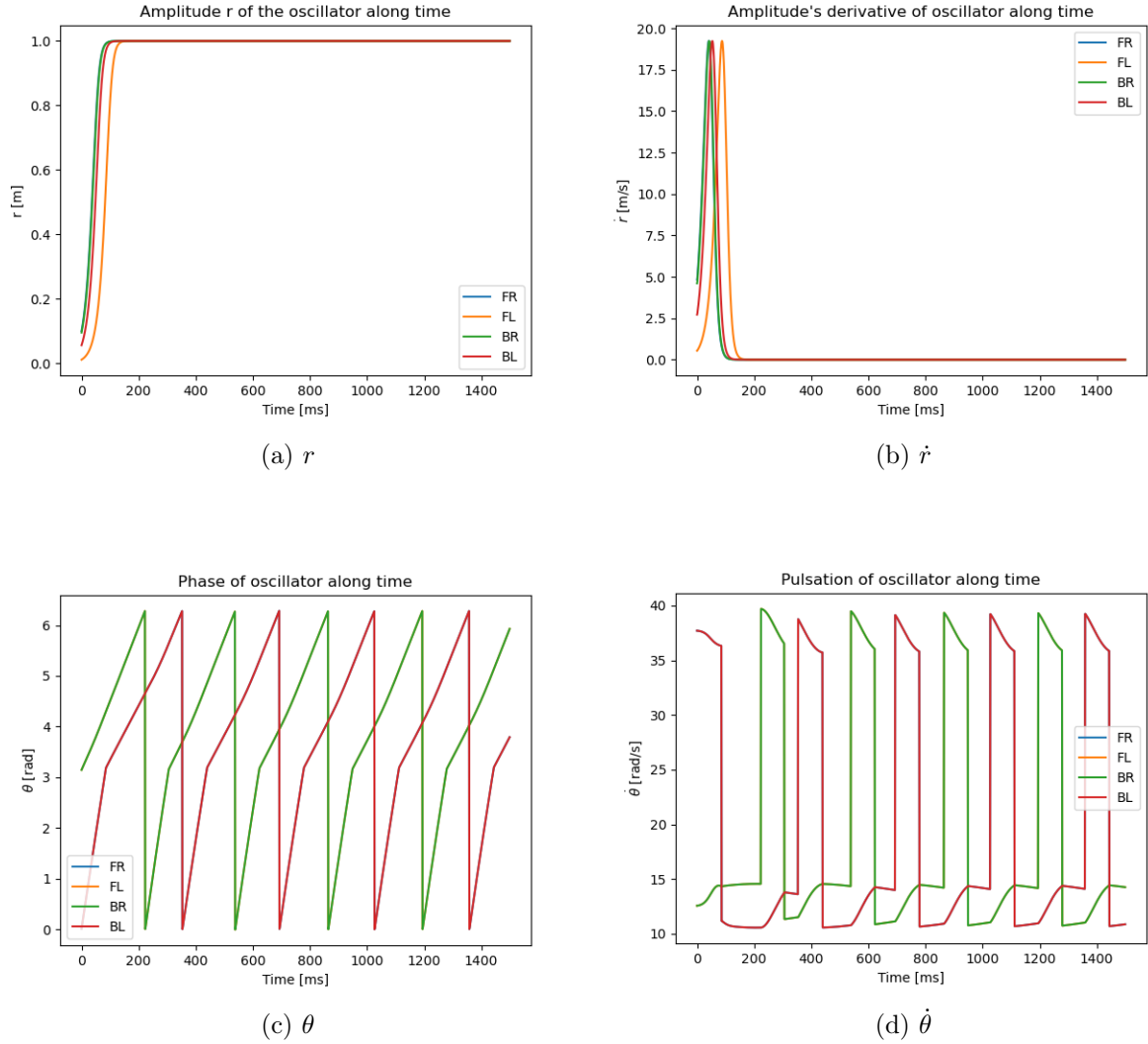
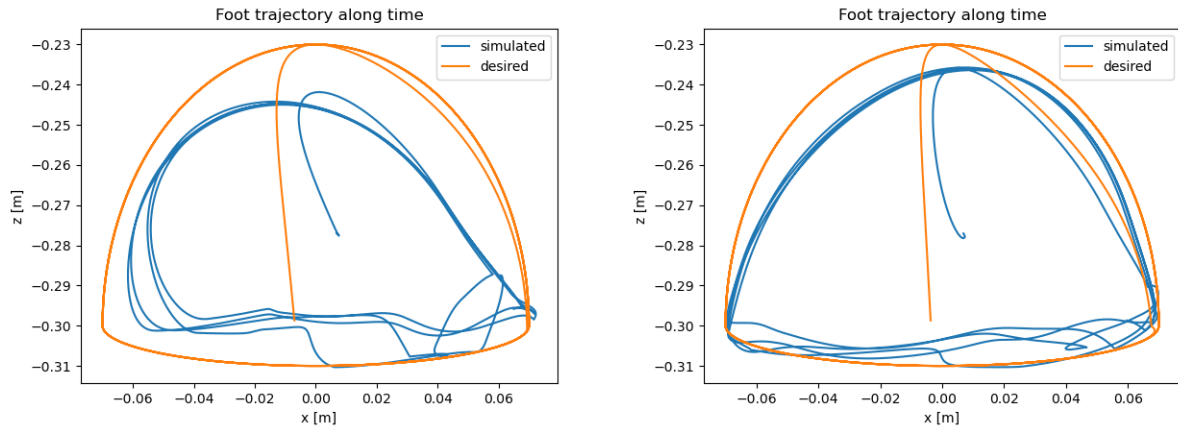


FIGURE 2 – Evolution of the CPG state along time for a trot gait

### 3.2.2 Foot trajectory

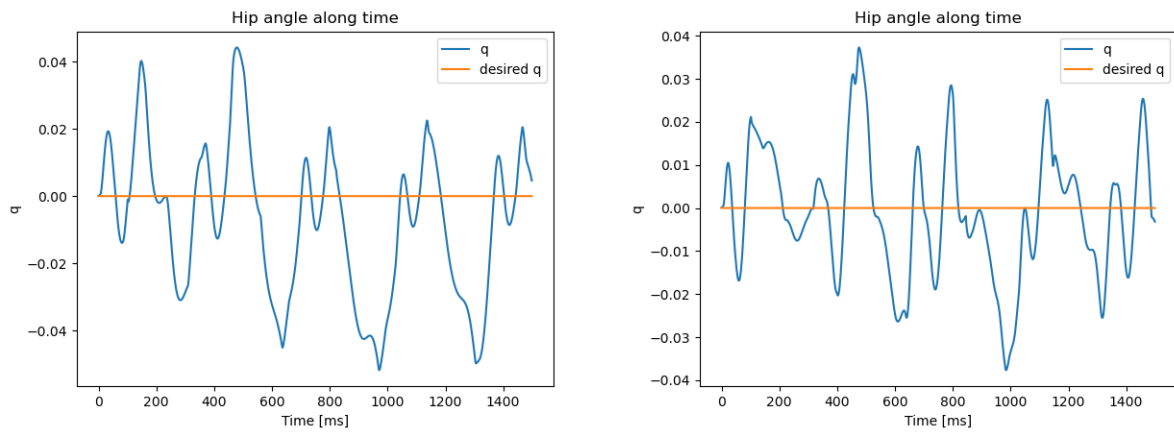


(a) Without cartesian PD

(b) With cartesian PD

FIGURE 3 – Plot of the foot trajectory with or without Cartesian PD

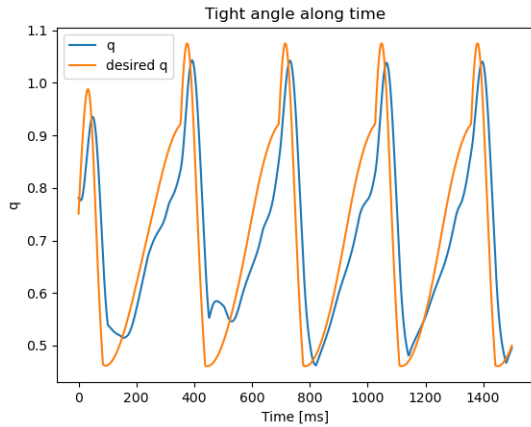
### 3.2.3 Joint angles trajectory



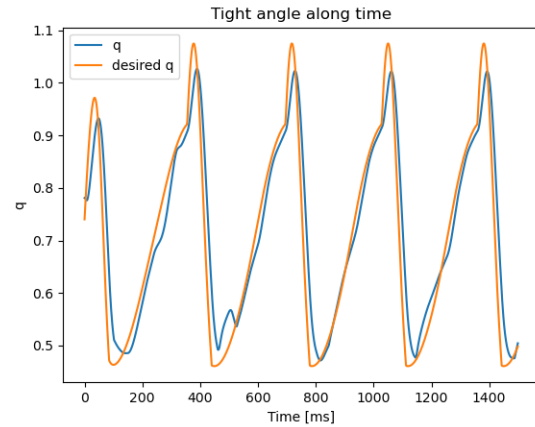
(a) Without cartesian PD

(b) With cartesian PD

FIGURE 4 – Plot of the hip trajectory with or without Cartesian PD (Front right leg)

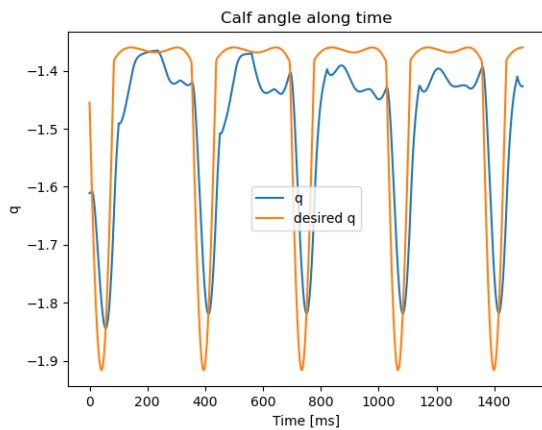


(a) Without cartesian PD

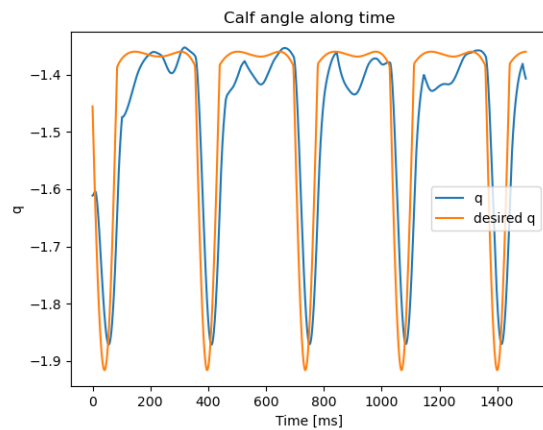


(b) With cartesian PD

FIGURE 5 – Plot of the thigh trajectory with or without Cartesian PD (Front right leg)



(a) Without cartesian PD



(b) With cartesian PD

FIGURE 6 – Plot of the calf trajectory with or without Cartesian PD (Front right leg)

### 3.2.4 Discussion of hyperparameters tuning

To obtain these performances, we found necessary to tune some hyperparameters on the used gait, such as the frequencies in swing or stance phase, the ground clearance or penetration and finally the desired step length. Other parameters we had to tune were the controller gains (for each joint and cartesian PD controllers) :

- The frequencies need to be high enough to avoid the robot to fail during the swing phase which is unstable, but not too high to let the robot grip to the floor and hence go forward.
- The ground clearance, ground penetration and desired step length are increased until the centroidal momentum is too big and the robot fails. That is the reason why we had to limit these hyperparameters.
- Finally the controller gains are tuned to allow the leg to follow the desired trajectory as closely as possible.



For the "TROT" gait, here are the tuned hyperparameters values :

$$\left\{ \begin{array}{l} \omega_{swing} = 12\pi \text{ rad} \cdot \text{s}^{-1} \\ \omega_{stance} = 4\pi \text{ rad} \cdot \text{s}^{-1} \\ \text{ground\_clearance} = 0.07 \text{ m} \\ \text{ground\_penetration} = 0.01 \text{ m} \\ \text{des\_step\_len} = 0.07 \text{ m} \\ Kp_{joint} = \text{np.diag}([100]*3) \\ Kd_{joint} = \text{np.diag}([2]*3) \\ Kp_{cartesian} = \text{np.diag}([1200]*3) \\ Kd_{cartesian} = \text{np.diag}([1]*3) \end{array} \right. , \quad \Phi_{\text{trot}} = 2\pi \cdot \begin{bmatrix} 0 & -0.5 & -0.5 & 0 \\ 0.5 & 0 & 0 & 0.5 \\ 0.5 & 0 & 0 & 0.5 \\ 0 & -0.5 & -0.5 & 0 \end{bmatrix} . \quad (1)$$

As the result of the above hyperparameters, here are the performances obtained by the simulation :

$$\left\{ \begin{array}{l} \bar{v}_{low} = 0.35 \text{ m} \cdot \text{s}^{-1} \\ \bar{v}_{high} = 0.6 \text{ m} \cdot \text{s}^{-1} \\ D = 3 \\ CoT = 0.5 \end{array} \right. \quad (2)$$

where  $\bar{v}$  is the average speed during a 10 second simulation,  $D$  is the duty cycle/ratio and  $CoT$  the cost of transport.

### 3.2.5 Locomotion videos

The locomotion videos are available in the zip file and are named as TROT\_HIGH\_XXms.mp4 and TROT\_LOW\_XXms.mp4.

### 3.2.6 Discussion of controller extension

We could extend our current controller in order to handle for difficult terrains or unexpected situations. Indeed, so far the terrain has been assumed flat but in the real world it would limit drastically the range of application of our robot, therefore tackling more difficult terrain would be interesting by focusing on the foot placement taking into account the slope, ground texture, friction cone, etc. Moreover we could add another control layer performing reflexes. This would help tackle obstacles blocking a foot and prevent the robot from falling as reflexes would on an animal.

### 3.3 Walk gait

#### 3.3.1 CPG states

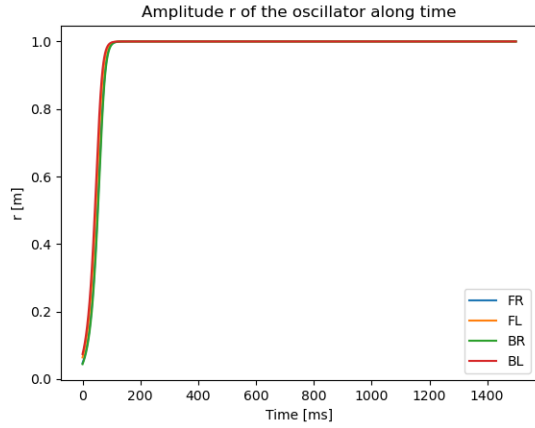
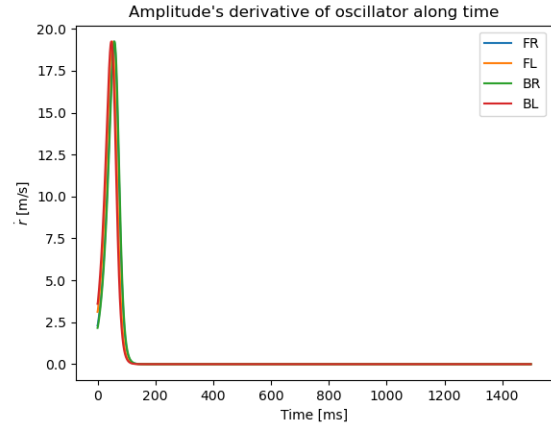
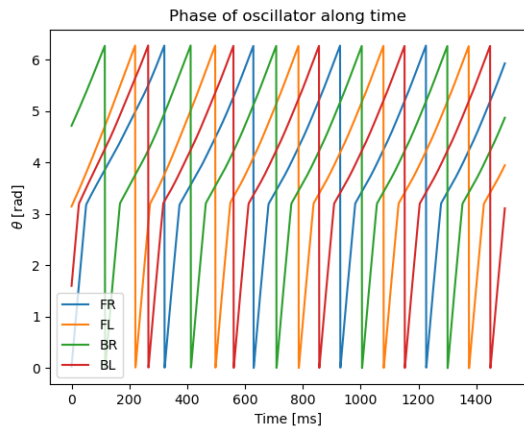
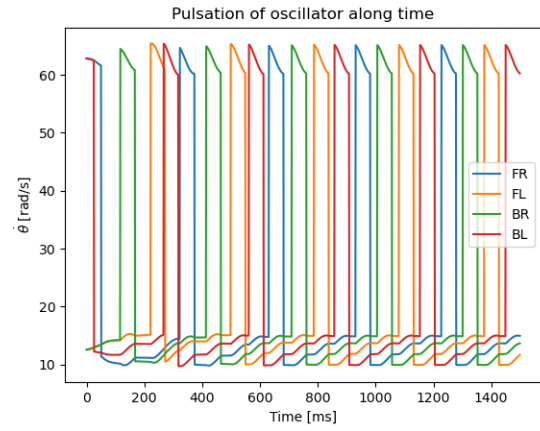
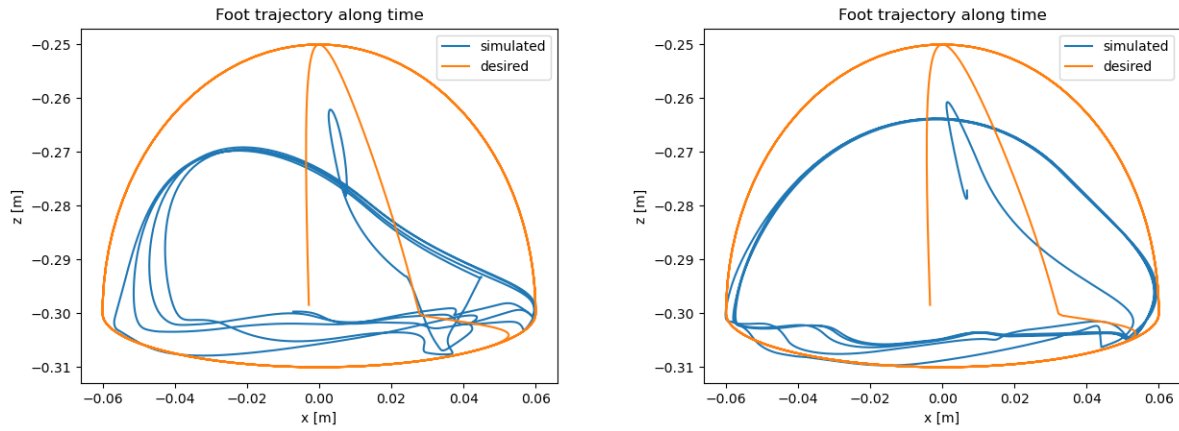
(a)  $r$ (b)  $\dot{r}$ (c)  $\theta$ (d)  $\dot{\theta}$ 

FIGURE 7 – Evolution of the CPG state along time for a walk gait

### 3.3.2 Foot trajectory

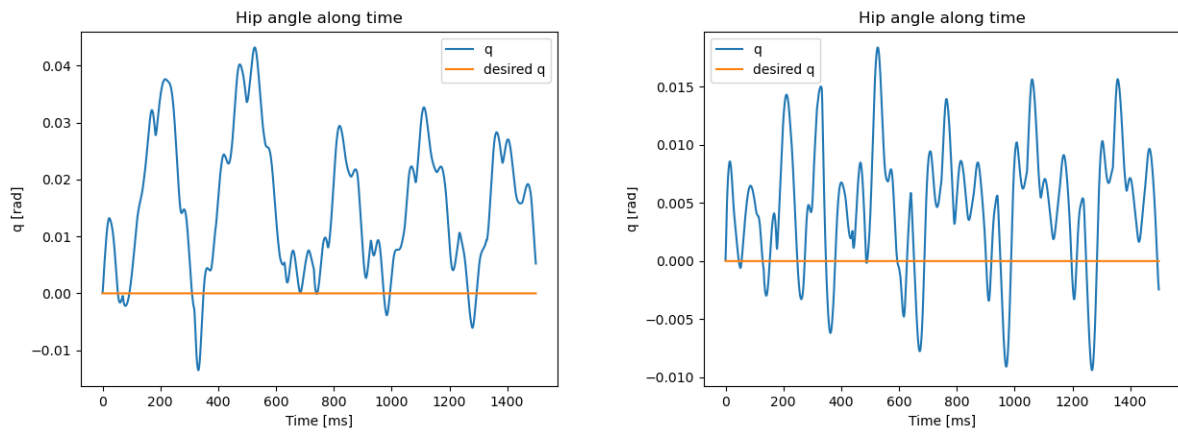


(a) Without cartesian PD

(b) With cartesian PD

FIGURE 8 – Plot of the foot trajectory with or without Cartesian PD

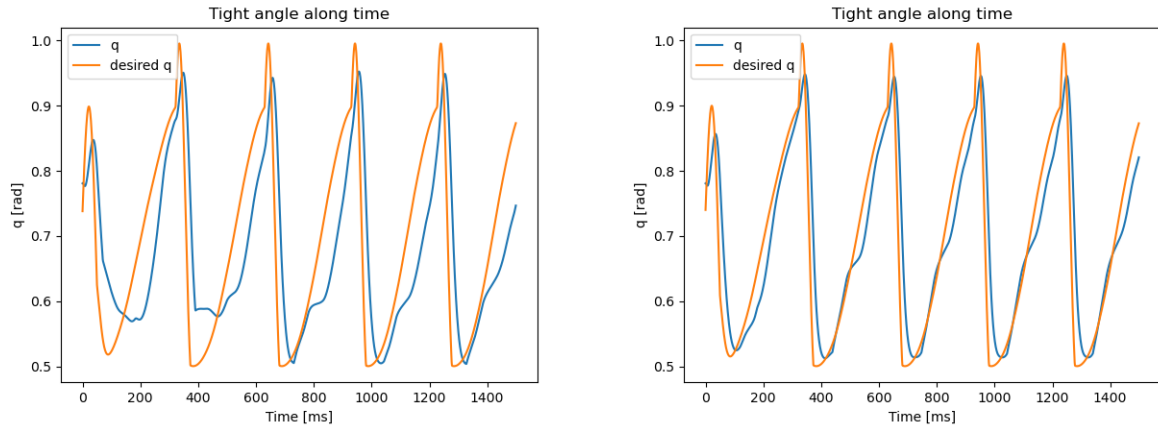
### 3.3.3 Joint angles trajectory



(a) Without cartesian PD

(b) With cartesian PD

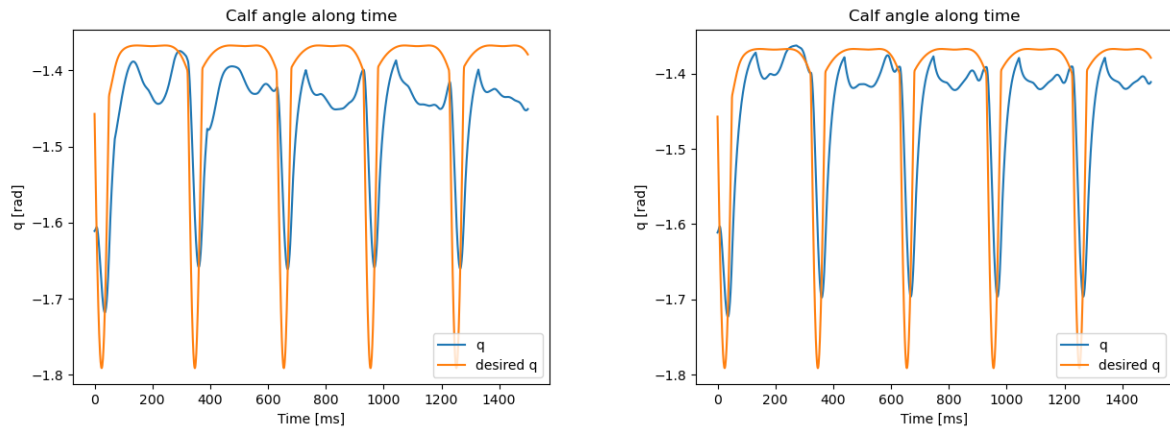
FIGURE 9 – Plot of the hip trajectory with or without Cartesian PD (Front right leg)



(a) Without cartesian PD

(b) With cartesian PD

FIGURE 10 – Plot of the thigh trajectory with or without Cartesian PD (Front right leg)



(a) Without cartesian PD

(b) With cartesian PD

FIGURE 11 – Plot of the calf trajectory with or without Cartesian PD (Front right leg)

### 3.3.4 Hyperparameters tuning

For the "WALK" gait, here are the tuned hyperparameters values :

$$\left\{ \begin{array}{l} \omega_{swing} = 10 \cdot 2\pi \text{rad} \cdot \text{s}^{-1} \\ \omega_{stance} = 2 \cdot 2\pi \text{rad} \cdot \text{s}^{-1} \\ \text{ground\_clearance} = 0.05\text{m} \\ \text{ground\_penetration} = 0.01\text{m} \\ \text{des\_step\_len} = 0.08\text{m} \\ Kp\_joint = \text{np.diag}([100]*3) \\ Kd\_joint = \text{np.diag}([2]*3) \\ Kp\_cartesian = \text{np.diag}([1200]*3) \\ Kd\_cartesian = \text{np.diag}([10]*3) \end{array} \right. , \quad \Phi_{walk} = 2\pi \cdot \begin{bmatrix} 0 & -0.5 & -0.25 & 0.25 \\ 0.5 & 0 & 0.25 & 0.75 \\ 0.25 & -0.25 & 0 & 0.5 \\ -0.25 & -0.75 & -0.5 & 0 \end{bmatrix} . \quad (3)$$

As the result of the above hyperparameters, here are the performances obtained by the simulation :

$$\begin{cases} \bar{v}_{low} = 0.46 \text{ m} \cdot \text{s}^{-1} \\ \bar{v}_{high} = 0.64 \text{ m} \cdot \text{s}^{-1} \\ D = 5 \\ CoT = 0.75 \end{cases} \quad (4)$$

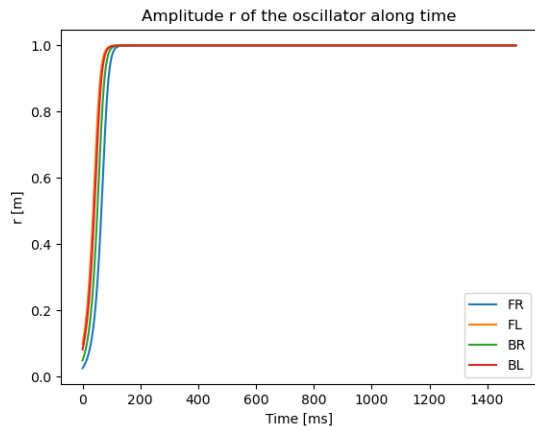
where  $\bar{v}$  is the average speed during a 10 second simulation,  $D$  is the duty cycle/ratio and  $CoT$  the cost of transport.

### 3.3.5 Locomotion videos

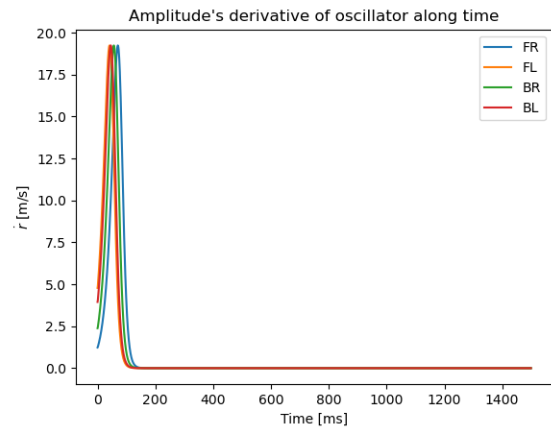
The locomotion videos are available in the zip file and are named as WALK\_HIGH\_XXms.mp4 and WALK\_LOW\_XXms.mp4.

## 3.4 Pace gait

### 3.4.1 CPG states



(a)  $r$



(b)  $\dot{r}$

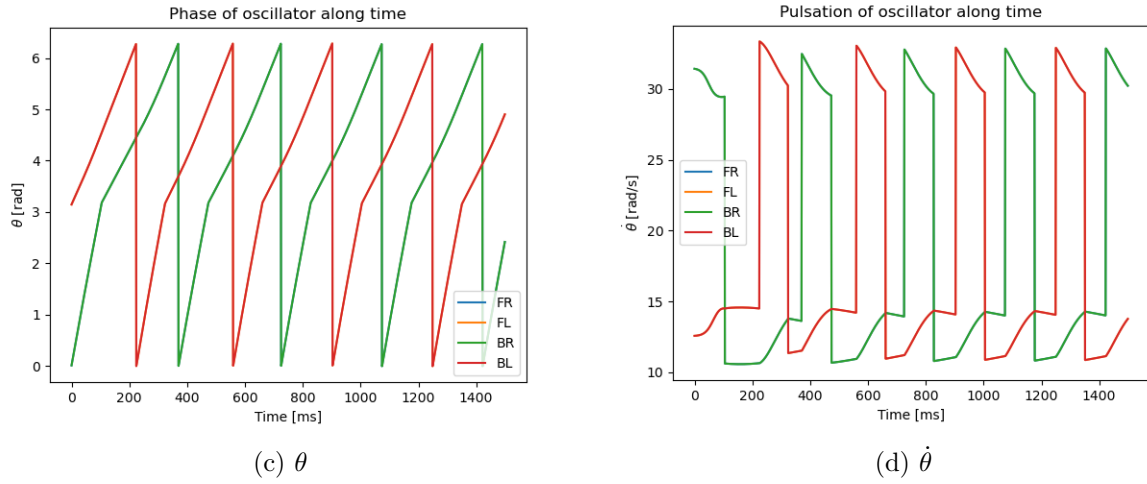


FIGURE 12 – Evolution of the CPG state along time for a pace gait

### 3.4.2 Foot trajectory

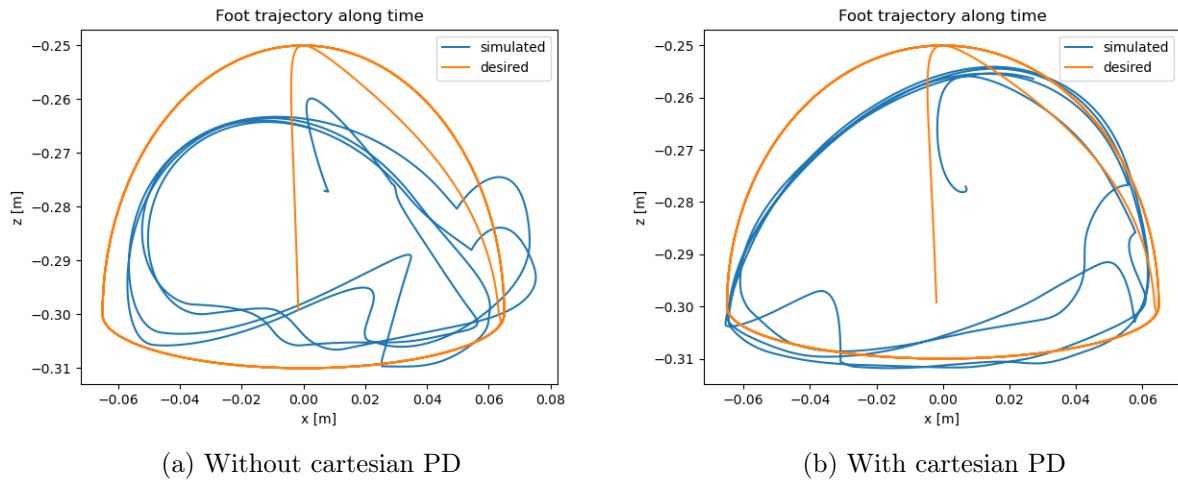
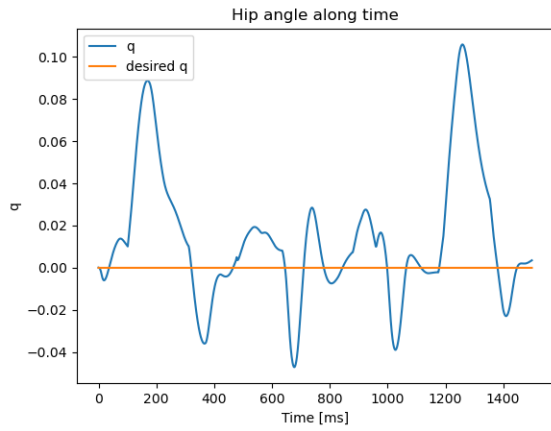
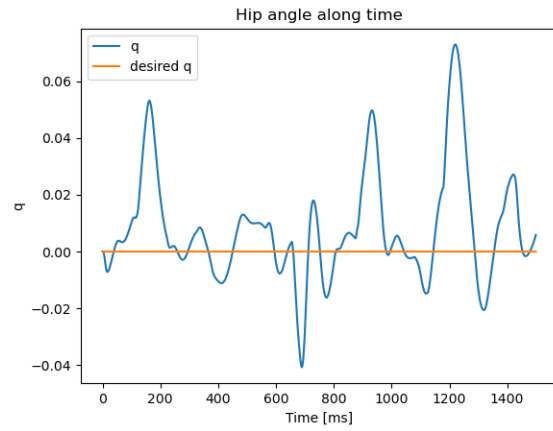


FIGURE 13 – Plot of the foot trajectory with or without Cartesian PD

### 3.4.3 Joint angles trajectory

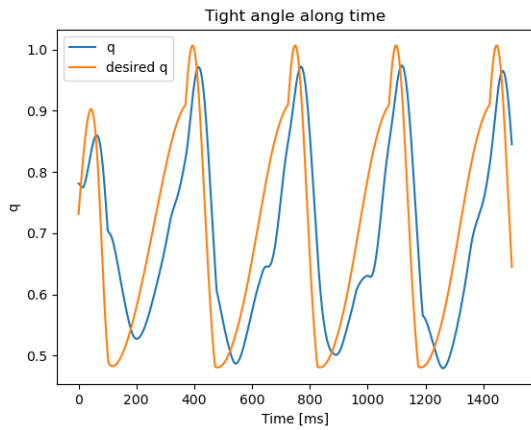


(a) Without cartesian PD

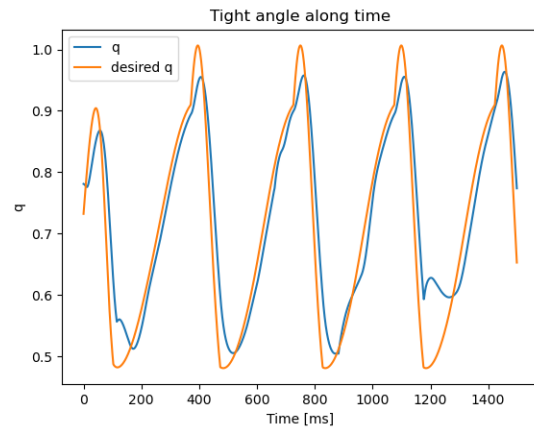


(b) With cartesian PD

FIGURE 14 – Plot of the hip trajectory with or without Cartesian PD (Front right leg)

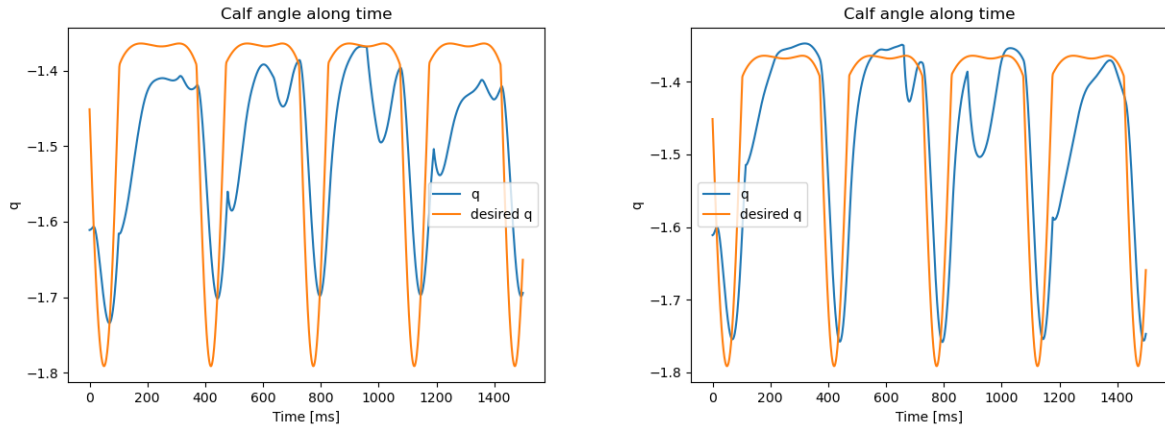


(a) Without cartesian PD



(b) With cartesian PD

FIGURE 15 – Plot of the thigh trajectory with or without Cartesian PD (Front right leg)



(a) Without cartesian PD

(b) With cartesian PD

FIGURE 16 – Plot of the calf trajectory with or without Cartesian PD (Front right leg)

### 3.4.4 Hyperparameters tuning

For the "PACE" gait, here are the tuned hyperparameters values :

$$\left\{ \begin{array}{l} \omega_{swing} = 10\pi \text{ rad} \cdot \text{s}^{-1} \\ \omega_{stance} = 4\pi \text{ rad} \cdot \text{s}^{-1} \\ \text{ground\_clearance} = 0.05 \text{ m} \\ \text{ground\_penetration} = 0.01 \text{ m} \\ \text{des\_step\_len} = 0.065 \text{ m} \\ Kp\_joint = \text{np.diag}([70]*3) \\ Kd\_joint = \text{np.diag}([2]*3) \\ Kp\_cartesian = \text{np.diag}([1200]*3) \\ Kd\_cartesian = \text{np.diag}([1]*3) \end{array} \right. , \quad \Phi_{pace} = 2\pi \cdot \begin{bmatrix} 0 & -0.5 & 0 & -0.5 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & -0.5 & 0 & -0.5 \\ 0.5 & 0 & 0.5 & 0 \end{bmatrix} . \quad (5)$$

As the result of the above hyperparameters, here are the performances obtained by the simulation :

$$\left\{ \begin{array}{l} \bar{v}_{low} = 0.3 \text{ m} \cdot \text{s}^{-1} \\ \bar{v}_{high} = 0.54 \text{ m} \cdot \text{s}^{-1} \\ D = 2.5 \\ CoT = 0.46 \end{array} \right. \quad (6)$$

where  $\bar{v}$  is the average speed during a 10 second simulation,  $D$  is the duty cycle/ratio and  $CoT$  the cost of transport.

### 3.4.5 Locomotion videos

The locomotion videos are available in the zip file and are named as PACE\_HIGH\_XXms.mp4 and PACE\_LOW\_XXms.mp4.



### 3.5 Bound gait

#### 3.5.1 CPG states

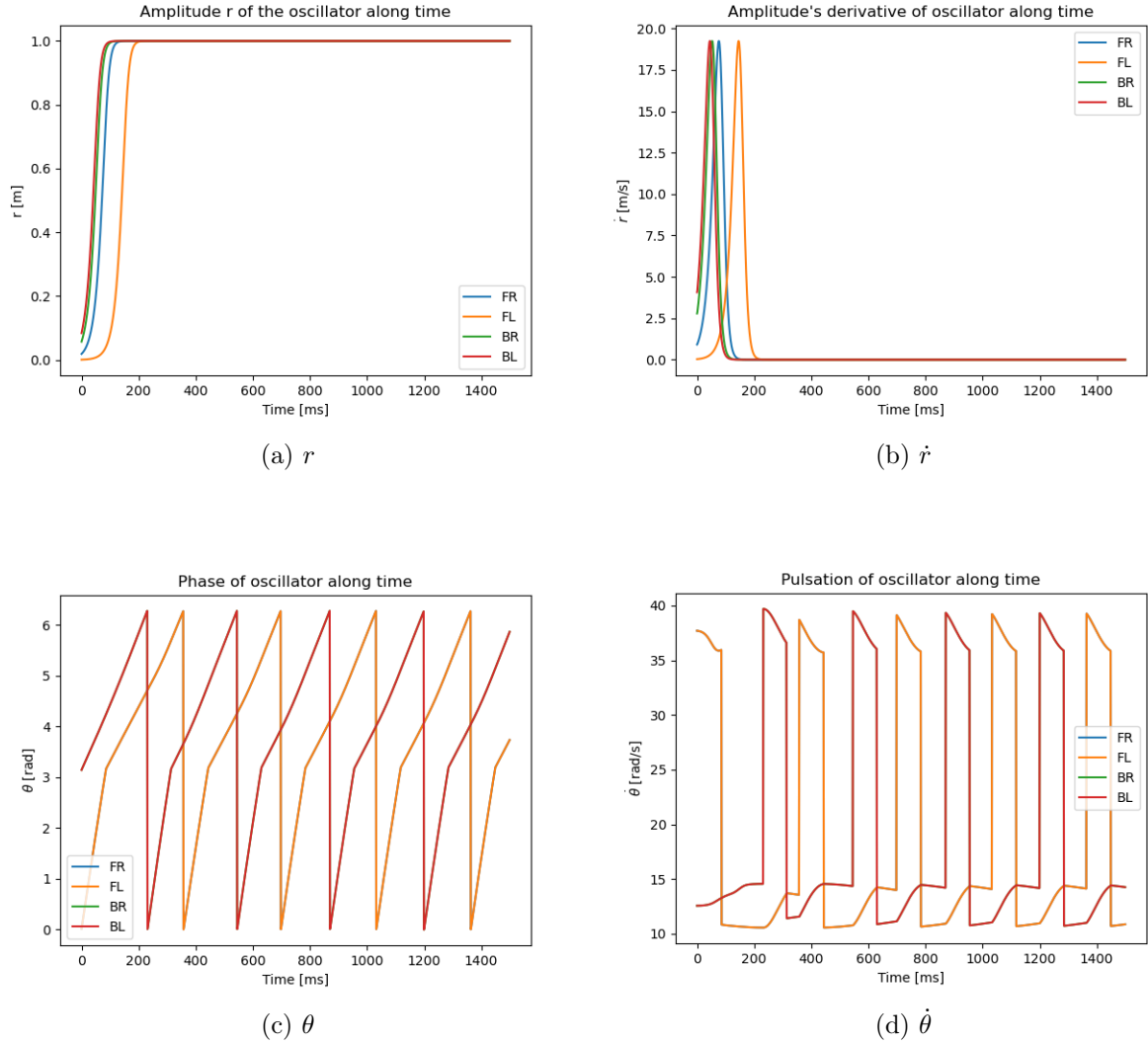
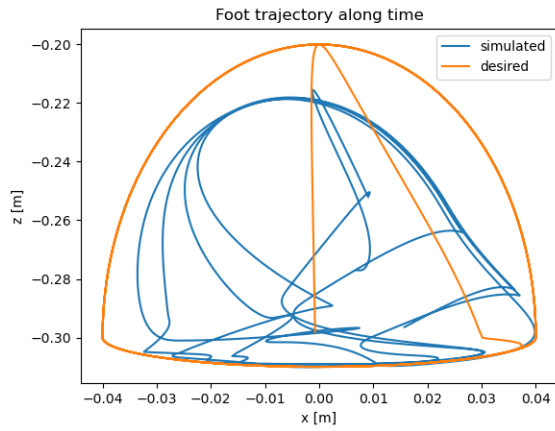
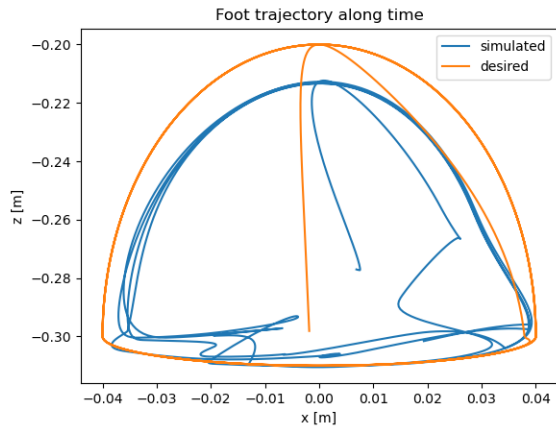


FIGURE 17 – Evolution of the CPG state along time for a bound gait

### 3.5.2 Foot trajectory



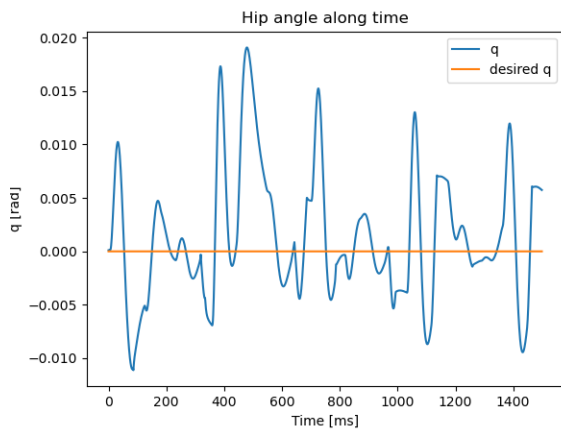
(a) Without cartesian PD



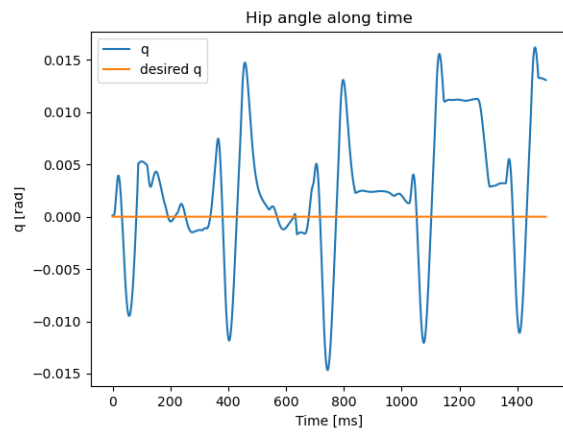
(b) With cartesian PD

FIGURE 18 – Plot of the foot trajectory with or without Cartesian PD

### 3.5.3 Joint angles trajectory

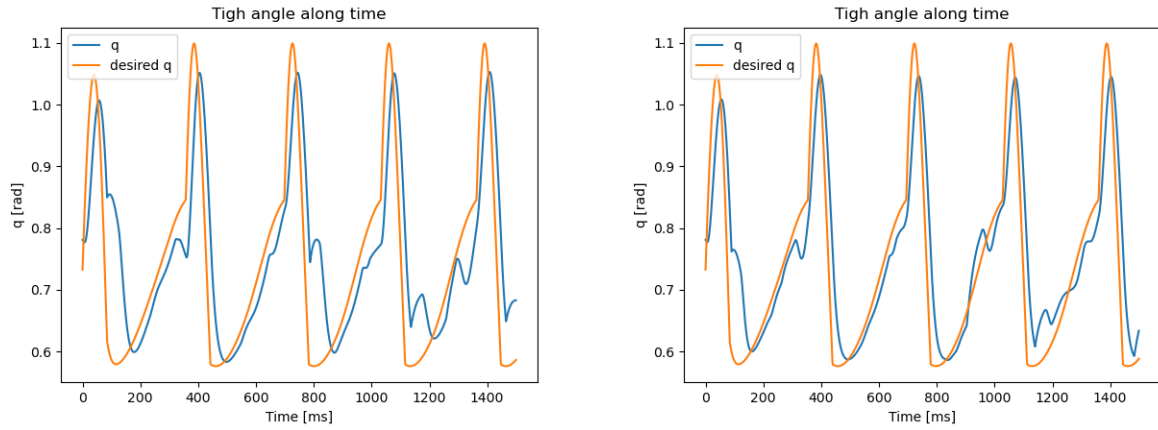


(a) Without cartesian PD



(b) With cartesian PD

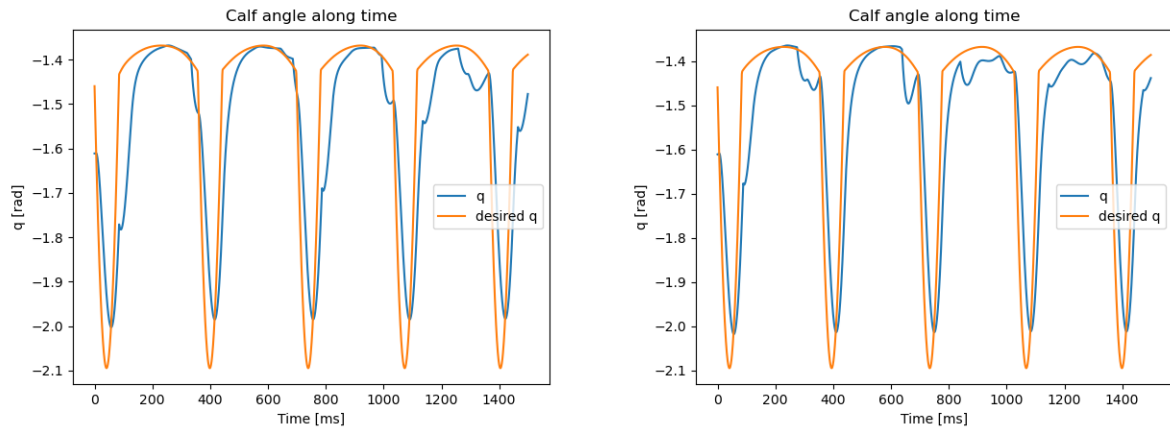
FIGURE 19 – Plot of the hip trajectory with or without Cartesian PD (Front right leg)



(a) Without cartesian PD

(b) With cartesian PD

FIGURE 20 – Plot of the thigh trajectory with or without Cartesian PD (Front right leg)



(a) Without cartesian PD

(b) With cartesian PD

FIGURE 21 – Plot of the calf trajectory with or without Cartesian PD (Front right leg)

### 3.5.4 Hyperparameters tuning

For the "BOUND" gait, here are the tuned hyperparameters values :

$$\left\{ \begin{array}{l} \omega_{swing} = 6 \cdot 2\pi \text{ rad} \cdot \text{s}^{-1} \\ \omega_{stance} = 2 \cdot 2\pi \text{ rad} \cdot \text{s}^{-1} \\ \text{ground\_clearance} = 0.1 \text{ m} \\ \text{ground\_penetration} = 0.01 \text{ m} \\ \text{des\_step\_len} = 0.04 \text{ m} \\ Kp_{joint} = \text{np.diag}([100]*3) \\ Kd_{joint} = \text{np.diag}([2]*3) \\ Kp_{cartesian} = \text{np.diag}([1200]*3) \\ Kd_{cartesian} = \text{np.diag}([10]*3) \end{array} \right. , \quad \Phi_{bound} = 2\pi \cdot \begin{bmatrix} 0 & 0 & -0.5 & -0.5 \\ 0 & 0 & -0.5 & -0.5 \\ 0.5 & 0.5 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 \end{bmatrix} . \quad (7)$$

As the result of the above hyperparameters, here are the performances obtained by the simulation :

$$\begin{cases} \bar{v}_{low} = 0.22\text{m} \cdot \text{s}^{-1} \\ \bar{v}_{high} = 0.33\text{m} \cdot \text{s}^{-1} \\ D = 3 \\ CoT = 1.12 \end{cases} \quad (8)$$

where  $\bar{v}$  is the average speed during a 10 second simulation,  $D$  is the duty cycle/ratio and  $CoT$  the cost of transport.

### 3.5.5 Locomotion videos

The locomotion videos are available in the zip file and are named as BOUND\_HIGH\_XXms.mp4 and BOUND\_LOW\_XXms.mp4.

## 4 Deep Reinforcement Learning

We need to design a Markov Decision Process by using Deep Reinforcement Learning.

### 4.1 Action space

For the action space, we modulate our actions to match with the tracking of a CPG oscillator. Using CPG instead of Cartesian PD or Joint PD seemed more intuitive because it would mean that we try to teach the robot the component of a periodic movement, namely running or trotting, by tuning the  $\mu$ 's and the  $\omega$ 's rather than making the environment learn all the positions along the feet trajectories. This can be seen on the training curves because training using CPG converges much faster than using Cartesian PD. The trained policy looks quite smooth.

### 4.2 Observation space

For the observation space, we want the robot to be aware of his base states (position, velocity, orientation and angular velocity) so that it is aware of whether it is well aligned and moving in the right direction or not. The motors states (angle, velocity and torque) are also added in the observation space to give some knowledge on the foot dynamic. Moreover, informations on feet (Normal forces, In contact booleans, number of valid/invalid contact points) were given for the robot to understand which foot is in contact with the ground and if it has invalid collisions to ensure stability and feasibility. Finally, we added while tracking a CPG mode the CPG parameters  $(r, \dot{r}, \theta, \dot{\theta})$ . By including these observations in the observation space the robot has a broad knowledge about its states so that it can learn more efficiently a certain gait. The following table summarizes the upper and lower limit given to each observation.

| Observation                      | Upper bound                                          | Lower bound                                           |
|----------------------------------|------------------------------------------------------|-------------------------------------------------------|
| Motor angles                     | $[0.2, \frac{\pi}{4} + 0.4, -\frac{\pi}{2} + 0.4]*4$ | $[-0.2, \frac{\pi}{4} - 0.4, -\frac{\pi}{2} - 0.4]*4$ |
| Motor velocities                 | $[21]*12$                                            | $[-21]*12$                                            |
| Motor torques                    | $[33.5]*12$                                          | $[-33.5]*12$                                          |
| Base orientation                 | $[1]*4$                                              | $[-1]*4$                                              |
| Base roll/pitch/yaw              | $[\pi]*3$                                            | $[-\pi]*3$                                            |
| Base position                    | $[0.05]*3$                                           | $[-0.05]*3$                                           |
| Base velocities                  | $[1]*3$                                              | $[-1]*3$                                              |
| Base angular velocities          | $[\pi]*3$                                            | $[-\pi]*3$                                            |
| Number of valid contact points   | 4                                                    | 0                                                     |
| Number of invalid contact points | 4                                                    | 0                                                     |
| Feet normal forces               | $[50]*4$                                             | $[-50]*4$                                             |
| Feet in contact Boolean          | $[1]*4$                                              | $[0]*4$                                               |
| <b>For CPG-RL only</b>           |                                                      |                                                       |
| $r$                              | $[\sqrt{\mu_{RL}}]*4$                                | $[0]*4$                                               |
| $\theta$                         | $[2\pi]*4$                                           | $[0]*4$                                               |
| $\dot{r}$                        | $[\alpha \cdot \mu_{RL}]*4$                          | $[0]*4$                                               |
| $\dot{\theta}$                   | $[25]*4$                                             | $[-25]*4$                                             |

### 4.3 Reward function

The reward function we implemented to train the model is the following :

$$reward = vel_{reward} - 0.5 \cdot orientation_{penalty} - 0.5 \cdot drift\_height_{penalty} - 0.01 \cdot energy_{penalty} + 0.05 \quad (9)$$

with the terms presented below :

$$\begin{cases} vel_{reward} = \frac{(x_t - x_{t-1})}{T_{step} \cdot action_{repeat}} \\ orientation_{penalty} = |\Phi| + |\Theta| + |\Psi| \quad (\text{Roll, Pitch and Yaw}) \\ drift\_height_{penalty} = |y| + |z - 0.25| \\ energy_{penalty} = \sum |\tau \cdot \dot{q}| \end{cases} \quad (10)$$

To resume, we want to increase the forward velocity of the robot, and the time it stays upright (reason of the added constant 0.05). However we want to penalize the orientation of the robot to stay aligned with the forward direction. Moreover, we also penalize the movement along the y and z axis to prevent any drift or crouching, only the x axis is unconstrained. Finally, an energy penalty has been added to achieve energy efficient locomotion.

### 4.4 DRL algorithm

We tried to use both a Soft Actor-Critic algorithm and a Proximal Policy Optimization algorithm and came to the conclusion that PPO was more interesting in our case thanks to its low computation cost compared to SAC, making long teaching simulations possible, with very few differences in the resulting locomotion when comparing two trainings with the same number of iterations.

### 4.5 Environmental details

Some parameters we can change on the environments are the learning rate, the number of steps or the buffer size in SAC.

- If we put the learning rate too high or too low, the reward will never grow, hence we choose the value  $10^{-4}$ .
- The number of steps to run for each environment per update have to be high enough to be sure the robot will stay still during an acceptable simulation time. But if the value was too high it could take a very long training time. That's why we chose 4096.
- Finally the buffer size is the number of saved observations made by previous policies. Again, it's better in terms of precision to increase this value, but we have to limit it to get a reasonable computational time. In our case we put it at 300000.

## 4.6 Approach robustness

Our model need to be trained in noisy conditions to simulate the sensor noises. We used a noise standard deviation of 0.01 in the training process, so that the robot can be more resilient when it is in a situation of unexpected behaviour, like in the environment `test_env` or `competition_env`. Since in the real world the observation space is not ideal, we need to implement some good observers. Moreover, depending on how good the model applies to the real robot, it might have an impact on how good the controllers will behave in real conditions. But in case the previous conditions on the observers and controllers are respected, we think the robot should be able to walk on a flat terrain in a real situation.

## 4.7 Resulting control policy using PPO

### 4.7.1 Reward and Episode length

After training, these are the performances we obtain using PPO :

$$\begin{cases} \bar{v} = 1.75 \text{ m} \cdot \text{s}^{-1} \\ CoT = 0.38 \end{cases} \quad (11)$$

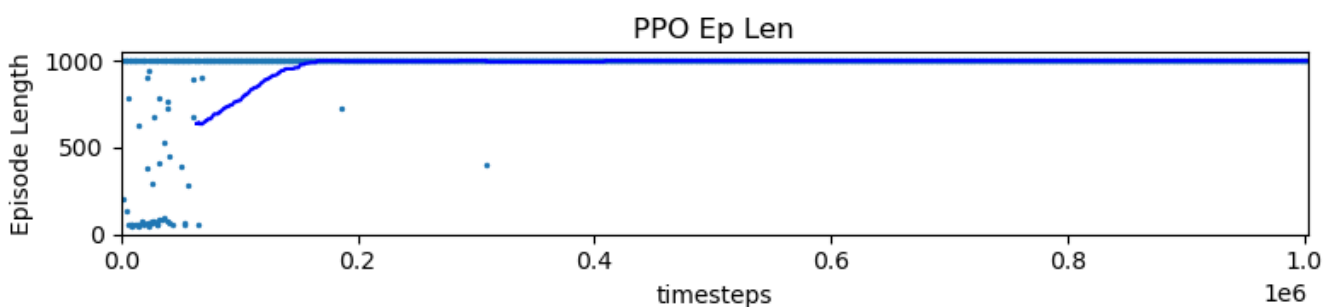


FIGURE 22 – Episode length using PPO

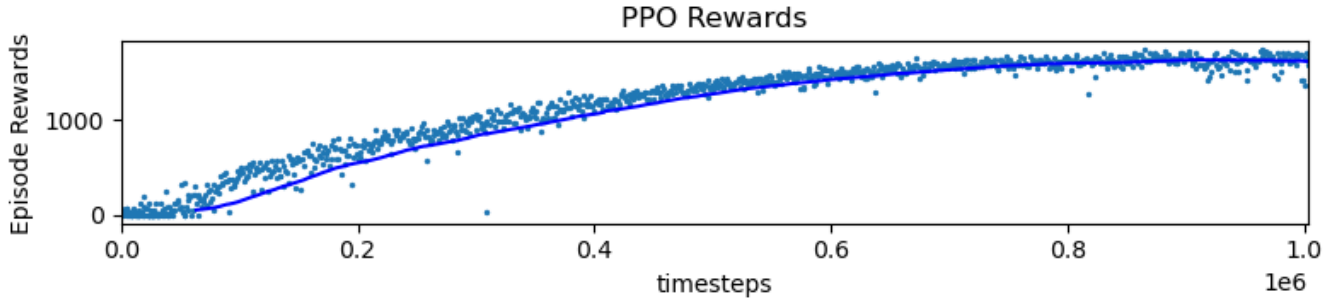


FIGURE 23 – Episode reward using PPO

#### 4.7.2 CPG states

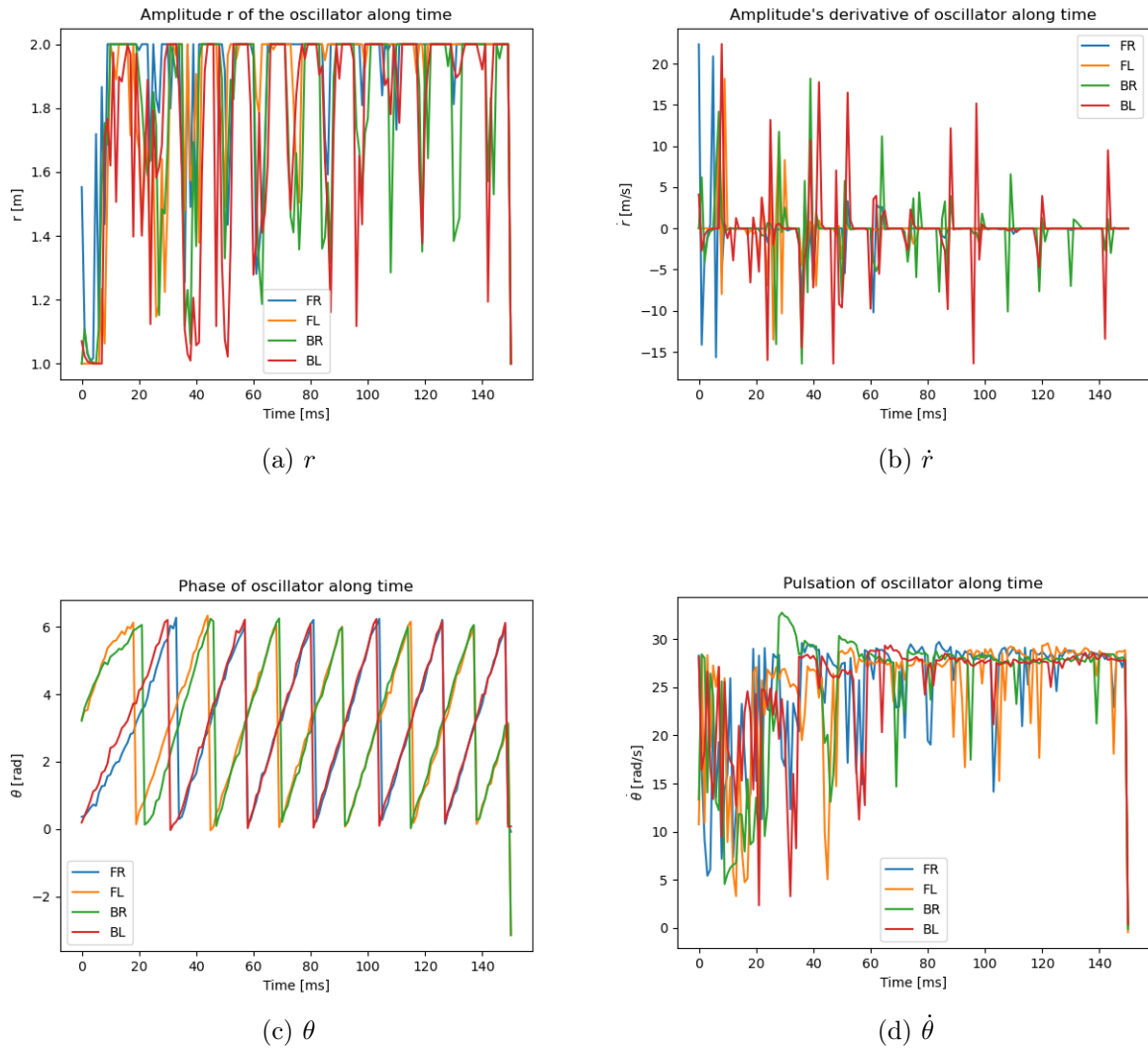


FIGURE 24 – Evolution of the CPG-RL states along time using PPO

### 4.7.3 Foot trajectory

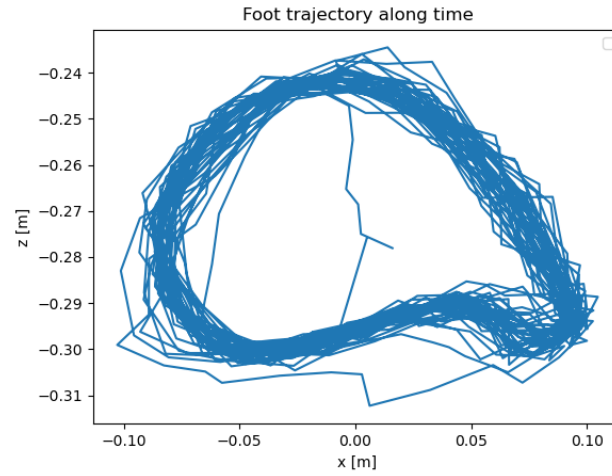
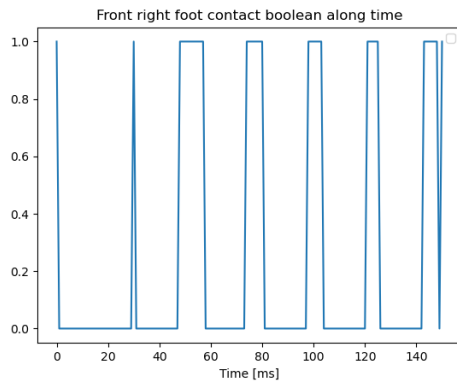
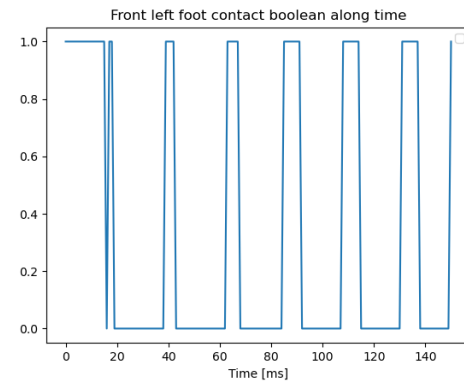


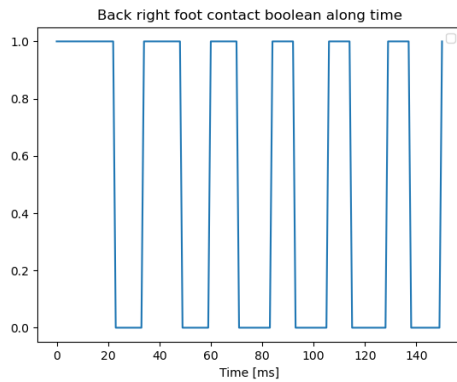
FIGURE 25 – Plot of the foot trajectory using PPO



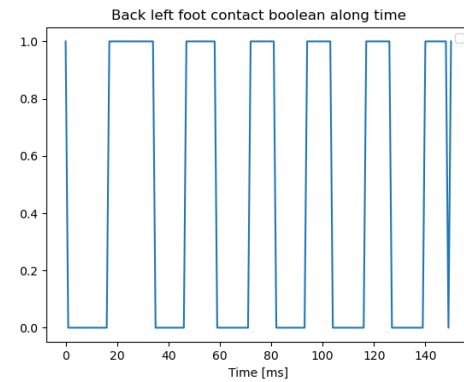
(a)



(b)



(c)



(d)

FIGURE 26 – Evolution of the foot contact Booleans along time using PPO



## 4.8 Resulting control policy using SAC

### 4.8.1 Reward and Episode length

After training, these are the performances we obtain using SAC :

$$\begin{cases} \bar{v} = 1.61 \text{ m} \cdot \text{s}^{-1} \\ CoT = 0.39 \end{cases} \quad (12)$$

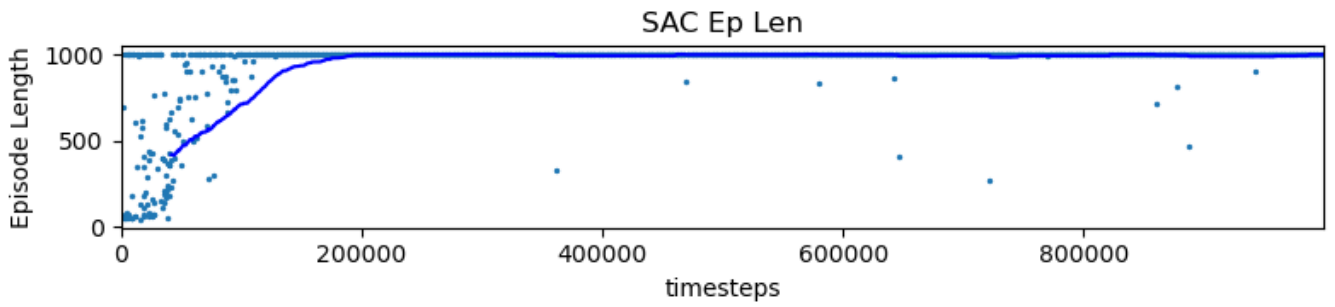


FIGURE 27 – Episode length using SAC

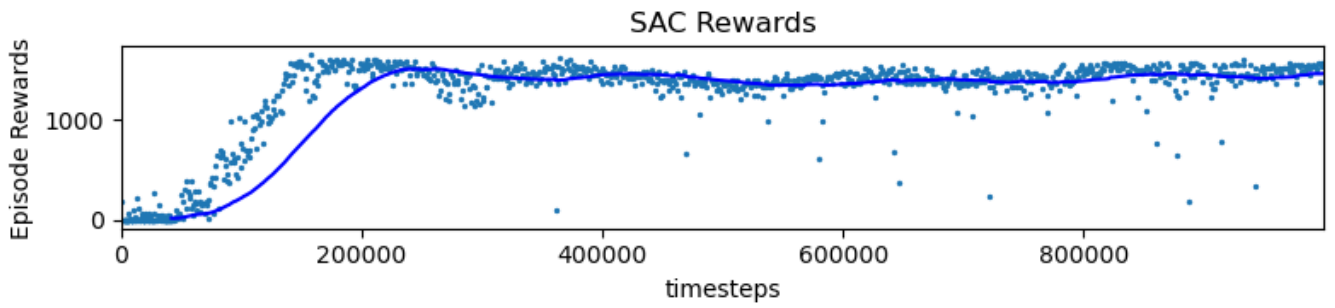


FIGURE 28 – Episode reward using SAC

## 4.8.2 CPG states

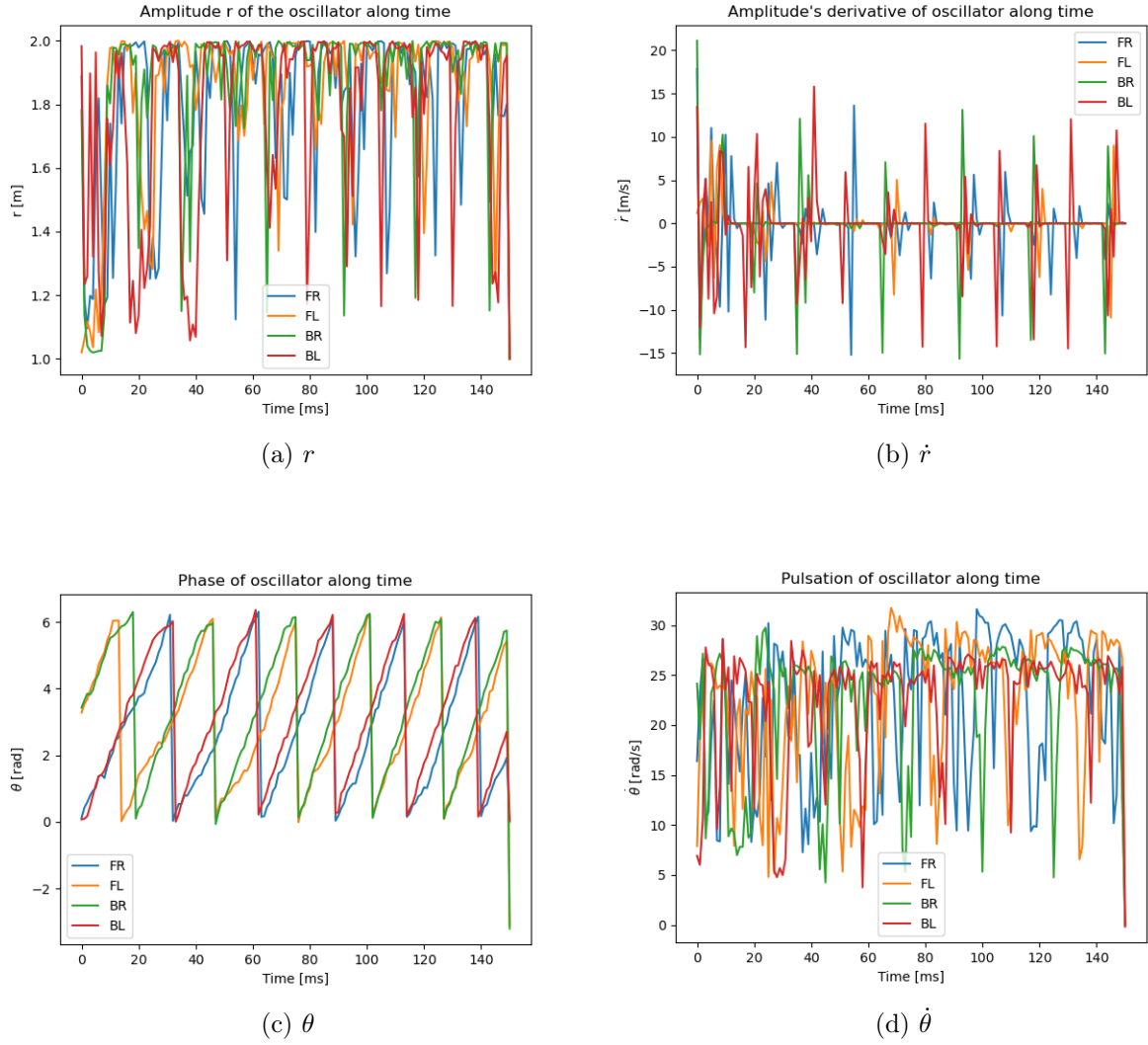


FIGURE 29 – Evolution of the CPG-RL states along time using SAC

### 4.8.3 Foot trajectory

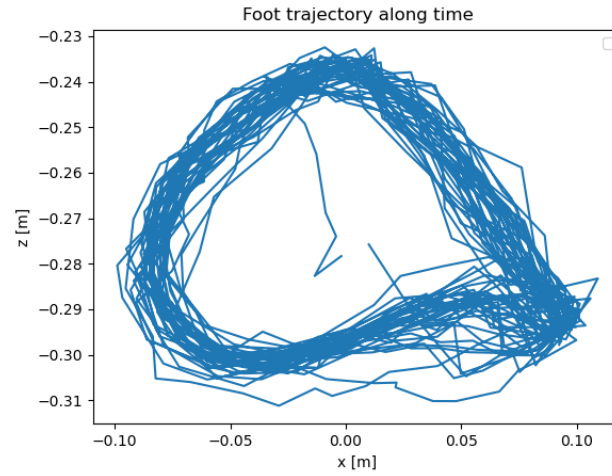
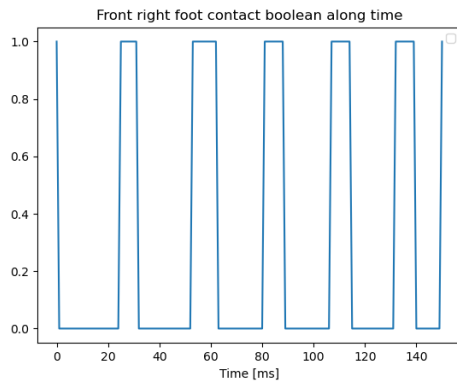
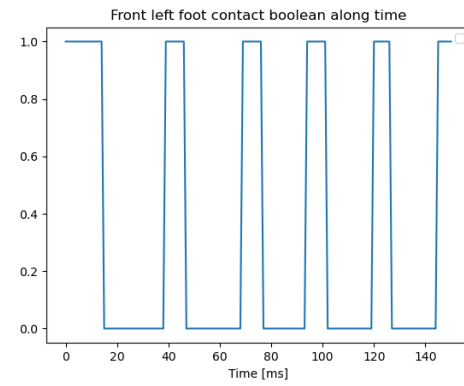


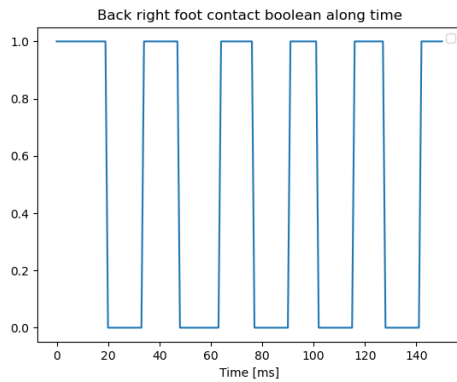
FIGURE 30 – Plot of the foot trajectory using SAC



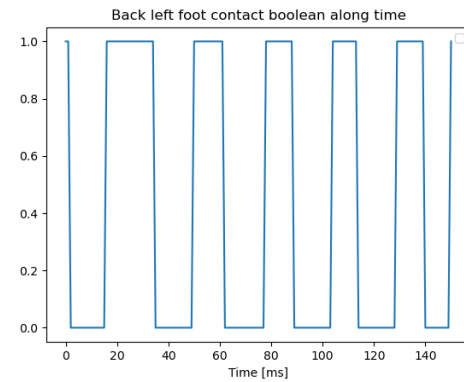
(a)



(b)



(c)



(d)

FIGURE 31 – Evolution of the foot contact Booleans along time using SAC

To have the robot deal with uneven terrain we could train in a different environment like `test_env` or any other having some randomness in the terrain generation. Doing so would result in a better control policy able to tackle a wide range of terrains. For the training in SAC and PPO we obtain a duty cycle of around 0.5 for the front legs and 1 for the back legs. Comparing this duty cycle with those obtained with CPG, we see that the robot spends a lot more time in swing than before which validates that our trained robot has a better and faster locomotion than when manually tuned.

#### 4.9 Locomotion videos

The locomotion videos are available in the zip file and are named as `PPO_CPG_TROT.mp4` and `SAC_CPG_TROT.mp4`.

### 5 Conclusion

The goal of this project was to simulate quadruped locomotion in PyBullet using CPG and using DRL. We could observe that the CPG method needed a lot of tuning in the feedback parameters and a lot of knowledge about the environment in order to get an acceptable stability and robustness. On the other hand, DRL needed a lot of knowledge about how the algorithms work in order to make a reward function giving us an acceptable motion.